



Earthheritance

Earthheritance — System Document

Cocopeat Manufacturing & Export Management

Generated: 22 September 2026

earthheritance-stock-manage.vercel.app

Earthheritance — System Document

Cocopeat manufacturing & export management system. Audience: developers and technical administrators. For day-to-day end-user instructions see [User Guide](#).

1. System overview

Earthheritance is a multi-user web application that tracks a cocopeat business end to end: buying raw cocopeat from suppliers, receiving it into stock lots, converting lots into finished products through production batches, selling export orders, planning shipments and dispatch, issuing commercial invoices, recording buyer receipts and supplier bills/payments, and reporting on everything — with a full audit trail and role-based access for six staff roles.

The app is **invitation-only**: there is no public sign-up. Every state change is written through a vetted database function (RPC) that checks the caller's permissions and records an audit event; the application never issues direct SQL writes to business tables.

2. Technology stack

Layer	Choice
Frontend / server	Next.js 16 App Router, React 19, TypeScript (strict)
Rendering	React Server Components; mutations via React Server Actions
Database	PostgreSQL 16 on Supabase, Row Level Security (RLS)
Auth	Supabase GoTrue (email/password, invitation-only)
File storage	Supabase Storage — private attachments bucket
Realtime	Supabase Realtime on key tables (live refresh badge)
Testing	Vitest + PGlite (in-process Postgres applying real migrations)
Hosting	Vercel (production branch main)

3. Architecture

Browser

| HTML (RSC) / form posts (Server Actions)



Next.js on Vercel

└ src/app/(workspace)/...	pages (server components, permission-gated)
└ src/app/actions.ts	ALL mutations: validate → RPC → revalidate
└ src/lib/records.ts	typed reads (RLS-scoped select-only queries)
└ src/components/...	UI (client islands: forms, modals, tables)
PostgREST (user JWT)	service-role key (server-only, narrow use)



Supabase Postgres

Supabase Auth / Storage

└ base tables: RLS select-only, no browser writes
└ public.app_* functions: SECURITY DEFINER – authorize + write + audit
└ reporting views: security_invoker=true (inherit caller's RLS)

Key rules:

- **Reads** go through `src/lib/records.ts`, each guarded by `guard(permission)`; RLS then scopes rows to the signed-in user.
- **Writes** go through `SECURITY DEFINER` RPCs (`app_create_supplier`, `app_confirm_purchase`, `app_dispatch_shipment`, ...). Every RPC re-checks `public.app_has_permission(...)` inside the database — the server action check is defense in depth, not the only gate.
- The service-role client (`createAdminSupabase`) is used in exactly three places: sending invitations, verifying/resetting passwords via the Auth admin API, and confirming/mounting uploaded Storage objects.
- `records.ts` starts with `import "server-only"`; client components may only `import type` from it (vitest aliases `server-only` to a stub).

4. Repository layout

src/	
app/	
(workspace)/	authenticated area (layout = WorkspaceShell)
page.tsx	Overview dashboard
modules/...	business modules (see §5)
team/ roles/ settings/ setup/ account/password/	
attachments/[id]/	signed-URL download route (RLS-gated)
login/ auth/callback/	anonymous auth flow
components/	UI; forms.tsx, currency.tsx, *-ui.tsx per module
lib/	
permissions.ts	role/permission catalog (shared with DB)
modules.ts	navigation module registry
records.ts	typed data layer (server-only)
validation.ts	zod schemas for every server action
workspace.ts	getActor / requirePermission / getStaff
supabase/	client factories (browser / server / admin)
supabase/migrations/	0001...0009, applied in order (see §6)
tests/	Vitest + PGlite integration suites
scripts/seed-demo.mjs	opt-in demo data seeder
docs/	this document + user guide

5. Modules and routes

Route	Purpose	Gate
/	Overview dashboard (KPIs per permission)	any active staff
/modules/suppliers , /modules/suppliers/[id]	Supplier directory + profile	suppliers.view
/modules/buyers , /modules/buyers/[id]	Buyer directory + profile	buyers.view
/modules/inventory	Stock on hand, lots, movements	inventory.view
/modules/inventory/items	Item catalogue (raw/finished)	inventory.manage
/modules/inventory/purchases[/id]	Purchase orders + goods receipts	inventory.view
/modules/inventory/adjustments	Stock adjustments (request → approve)	inventory.view
/modules/production[/id]	Production batches: inputs → output + wastage	production.view

Route	Purpose	Gate
/modules/exports[/id]	Export orders, reservations, shipments, invoices	exports.view
/modules/supplier-payments	Bills, payments, allocation, reversal register	finance.view
/modules/buyer-receipts	Export invoices and receipt register	finance.view
/modules/reports	7 reporting views + CSV export	reports.view
/modules/documents/invoice/[id]	Printable commercial invoice (letterhead)	exports.view
/modules/documents/packing/[id]	Printable packing list	exports.view
/modules/files/[entity]/[id]	Supporting-document upload/list per record	entity read permission
/attachments/[id]	Download: RLS check → 30 s signed URL	document owner check
/team	Invite staff, roles, activate/deactivate, password reset	users.manage
/roles	Edit role → permission assignments	roles.manage
/settings	Company profile, base currency, warehouse	settings.manage
/account/password	Self-service password change	any signed-in user
/setup	Guided administrator setup checklist	administrator

6. Data model (migrations 0001–0009)

Migration	Contents
0001_foundation	app_role enum, profiles, permission_catalog, role_permissions, company_settings, staff_invitations, audit_events; RLS base; app_has_permission, app_set_staff_access (last-admin guard), invitation RPCs + enrollment trigger, app_bootstrap_admin
0002_suppliers_stock	suppliers, items, purchases / purchase_lines, goods_receipts /lines, lots, stock_movements, stock_adjustments, supplier_payment_details; app_log_audit helper; supplier/item/purchase/receipt/adjustment RPCs

Migration	Contents
0003_production	production_batches , production_inputs ; consume-input + create-output + movements atomically; reversal rules; traceability
0004_exports_finance	buyers , export_orders /lines, shipments /lines, export_invoices /lines, supplier_bills , supplier_payments /allocations, buyer_receipts /allocations; FIFO reservations, dispatch, invoice/receipt/bill/payment RPCs; realtime publication
0005_reporting	7 security_invoker reporting views (stock, production summary, export sales lines, shipment schedule, overdue documents, unapplied balances, allocation ledgers)
0006_performance	10 FK/lookup indexes (benchmark-proven)
0007_register_references	Payment/receipt balance views expose method + reference for register filtering
0008_documents	documents table (entity whitelist, 10 MB, mime whitelist, path regex), app_document_read_ok / write_ok , app_register_document , app_remove_document , private attachments bucket + path-scoped storage policies
0009_staff_password_audit	app_log_staff_password_reset(uuid) — audit RPC for admin password resets (see §12)

Status enums (lifecycle states):

- purchase: draft → confirmed → partially_received → received , OR cancelled
- production batch: draft → posted → reversed
- export order: draft → confirmed → partially_shipped → shipped → closed , OR cancelled
- shipment: draft → ready → dispatched → delivered , OR cancelled
- stock adjustment: pending → approved | rejected
- payment/receipt ledger: recorded | reversed
- movement types: opening, receipt, adjustment_increase, adjustment_decrease, production_consumption, production_output, dispatch

7. Security model

- **Six roles** (app_role): administrator , manager , stores , production , export_sales , finance .
- **Permission catalog** (key → meaning): suppliers.view/manage , purchasing.manage , inventory.view/manage/approve , production.view/manage , buyers.view/manage , exports.view/manage , finance.view/manage , reports.view , audit.view , users.manage , roles.manage , settings.manage .
- Default assignments:

Role	Permissions
administrator	everything (short-circuits in <code>app_has_permission</code> ; cannot be edited away)
manager	all <code>*.view</code> + <code>inventory.approve</code> + <code>audit.view</code>
stores	<code>suppliers.view/manage</code> , <code>purchasing.manage</code> , <code>inventory.view/manage</code>
production	<code>inventory.view</code> , <code>production.view/manage</code>
export_sales	<code>buyers.view/manage</code> , <code>exports.view/manage</code> , <code>inventory.view</code>
finance	<code>suppliers.view</code> , <code>buyers.view</code> , <code>exports.view</code> , <code>finance.view/manage</code> , <code>reports.view</code>

- **RLS:** every business table is `select` -only for `authenticated` ; `INSERT/UPDATE/DELETE` is reserved for SECURITY DEFINER functions. Reporting views use `security_invoker=true` so they cannot leak beyond the caller's own RLS.
- **Session invalidation:** `profiles.security_revision` bumps on role, permission or activation changes, so cached permission sets go stale.
- **Last-admin guard:** `app_set_staff_access` (advisory-locked) refuses to demote or deactivate the final active administrator; the password-reset path refuses to target them as well (\$12).
- **Fail-closed:** without `NEXT_PUBLIC_SUPABASE_URL` + `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` the app runs in read-only preview mode; all writes respond "Connect Supabase first..."

8. Stock, lots and traceability

- One warehouse in v1. Stock per **item** is derived from `stock_movements` (`quantity_delta`); per **lot** from the same ledger filtered by `lot_id` . A conservation invariant is asserted in tests: item balance == $\sum$ lot balances, and reservations == open export-order balance.
- Purchasing: PO lines → goods receipts create/extend lots (raw material).
- Production: consumes raw lots (`production_consumption`), creates one finished lot (`production_output`), records wastage; a posted batch can be reversed only while its output lot is untouched.
- Exports: confirming an order **reserves** available finished lots FIFO; dispatch consumes reservations and writes `dispatch` movements; closing an order releases leftovers.
- Adjustments require a second person with `inventory.approve` .
- Traceability: finished lot → batch → input lots → supplier, surfaced on supplier/buyer profiles and reports.

9. Currency model

- `currency` is always a three-letter uppercase code; entry is via the shared `CurrencySelect` dropdown — **USD and LKR lead the list** (supplier payments settle in LKR; export orders/invoices in USD or other foreign currency).

- Every transaction stores its own `currency` , optional `exchange_rate` (manually entered, `numeric(16,6)`), and the original amount. Totals are **never summed across currencies**; registers, balances, statements and reports group and filter by currency.
- `company_settings.base_currency` is the reporting/home currency (Settings page dropdown).
- Suppliers and buyers carry a `preferred_currency` ; purchase and export order forms default the currency dropdown from the selected party.

10. Files and printable documents

Two distinct features:

1. **Printable documents** (`/modules/documents/...`): server-rendered commercial invoice and packing list with company letterhead, print CSS, generated live from data — no files stored.
2. **Supporting documents** (`/modules/files/[entity]/[id]`): real file uploads (pdf/jpg/png/webp/docx/xlsx, ≤10 MB) attached to one of eight entity types (`purchase` , `goods_receipt` , `export_order` , `shipment` , `export_invoice` , `supplier_bill` , `supplier_payment` , `buyer_receipt`).

Upload chain (defense in depth):

```
Browser validates (size/mime) → upload to private bucket path
<entity>/<record-uuid>/<uuid>-<safe-name> with the user's JWT
→ server action verifies the object exists (service-role list)
→ app_register_document RPC re-validates permission (write_ok per entity),
  path prefix, size, mime, entity existence → insert + audit
Download: /attachments/[id] → RLS select proves access → 30 s signed URL
Delete: app_remove_document → write_ok → delete storage object + row + audit
```

Storage policies derive access from `split_part(name, '/', 1)` via `app_document_read_ok` / `app_document_write_ok` , so the bucket stays private and entity-scoped.

11. Audit trail

`audit_events(actor_id, action, target, details jsonb)` — written inside the RPCs (never by direct browser insert). Visible on Overview to roles with `audit.view` . Notable actions include: `company.updated` , `staff.invitation_prepared` , `staff.invited` , `staff.access_updated` , `staff.password_reset` , `supplier.created/updated/active_updated` , `item.*` , `purchase.created/confirmed/cancelled` , `goods.received` , `adjustment.requested/approved/rejected` , batch post/reverse, order confirm/dispatch/deliver/close/cancel, invoice issued, receipt/payment recorded/allocated/reversed, `document.added` , `document.removed` , `role.permissions_updated` .

12. Authentication & team lifecycle

- **First administrator:** create an Auth user manually (Supabase dashboard, no email confirmation), then `select public.app_bootstrap_admin('<uuid>', 'Name');` — valid only while `profiles` is empty.
- **Invitations:** `/team` → `app_prepare_invitation` + service-role `inviteUserByEmail`; a trigger on `auth.users` inserts the profile with the invited role if the user appears within 24 h of a pending invitation. Requires SMTP configured in Supabase Auth.
- **Passwords:** minimum 12 characters. Self-service change at `/account/password` (revokes other sessions). Admin reset on `/team` (requires `users.manage`): the action checks the target profile via RLS, refuses to reset the last active administrator, calls Auth admin `updateUserById` (which signs the member out everywhere), then records the event via `app_log_staff_password_reset` — a dedicated SECURITY DEFINER audit RPC, because `app_log_audit` is deliberately **not** executable by browser sessions.
- **Deactivation** keeps all history and cuts access immediately (`app_is_active()` participates in RLS policies).

13. Environment & configuration

Required environment variables:

Name	Scope	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code>	public	Supabase project URL
<code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY</code>	public	anon key
<code>SUPABASE_SECRET_KEY</code>	server-only	service-role key (invites, storage checks, password reset)
<code>NEXT_PUBLIC_SITE_URL</code>	public	HTTPS origin used in invitation redirects

Supabase side: run migrations 0001→0009 **in order**; Auth → Site URL = app origin, redirect `<origin>/auth/callback`, disable public sign-up, minimum password length 12; SMTP configured for invitations.

14. Deployment

- GitHub: `https://github.com/Gihan55/earthheritance-stock`
- Branch flow: `feature/*` → `dev` → `preview` → `main`.
- Vercel project connected; **production branch** = `main`.
- Manual release gate: "Create deployments for connected branches" is disabled (Settings → Git) so only deliberate `main` activity or a **Deploy Hook** (`Invoke-RestMethod -Method Post <hook-url>`) triggers a production build; dashboard **Redeploy** re-publishes an existing commit.
- `NEXT_PUBLIC_*` values are baked at build time — changing them requires a rebuild/redeploy.

- Local: `npm run dev` (127.0.0.1:3000); without env vars the app opens in preview mode. `npm run seed:demo` populates a connected Supabase project with demo data (service-role ordering matters — see script comments).

15. Testing strategy

- `npm test` — 10 Vitest files, **113 tests**, all against PGLite instances that apply the real migration files (each suite keeps its own migration list; add new migrations to every list they must run under).
- Suites: foundation/RLS matrix, suppliers+stock, production, exports+finance, reporting views, documents, password-reset audit RPC, CSV, validation schemas, action wiring (static), plus one full-system lifecycle walkthrough asserting stock conservation and permission boundaries.
- Auth-only APIs (invitations, password resets) are tested at the RPC/audit boundary; the Auth admin calls themselves need a live project.
- `npm run build` + `npm run lint` + `npx tsc --noEmit` gate every release.

16. Troubleshooting catalog

Symptom	Cause / resolution
"The workspace is not connected to Supabase yet"	Missing <code>NEXT_PUBLIC_*</code> env vars on Vercel → set them and redploy
"Your staff access is not active"	No <code>profiles</code> row (bootstrap/invitation missing) or <code>is_active=false</code> → run <code>app_bootstrap_admin</code> or reactivate on <code>/team</code>
"The change could not be saved. Check your database setup..."	A migration was not run on the live project (compare §6 list), or an RPC grant is missing — check the browser network tab + Supabase logs
"Keep at least one active administrator"	Refuse to demote/deactivate the last admin — promote another first
"This email already has an account..." on invite	User exists in Auth → manage access instead of re-inviting
Document upload "The uploaded file was not found"	Storage object missing (failed upload) → retry upload; verify 0008 ran and the <code>attachments</code> bucket exists
Invitations not delivered	SMTP not configured in Supabase Auth, or <code>NEXT_PUBLIC_SITE_URL</code> unset